A Genetic Algorithm Approach to a General Category Project Scheduling Problem

Linet Özdamar

***Abstract*—A genetic algorithm (GA) approach is proposed for the general resource constrained project scheduling model, in which activities may be executed in more than one operating mode and renewable as well as nonrenewable resource constraints exist. Each activity operation mode has a different duration and requires different amounts of renewable and nonrenewable resources. The objective is the minimization of the project duration or makespan. The problem under consideration is known to be one of the most difficult scheduling problems, and it is hard to find a feasible solution for such a problem, let alone the optimal one.**

The GA approach described here incorporates problem-specific scheduling knowledge by an indirect chromosome encoding that consists of selected activity operating modes and an ordered set of scheduling rules. The scheduling rules in the chromosome are used in an iterative scheduling algorithm that constructs the schedule resulting from the chromosome. The proposed GA is denoted as a hybrid GA (HGA) approach since it is integrated with traditional scheduling tools and expertise specifically developed for the general resource constrained project scheduling problem. The results demonstrate that HGA approach produces near-optimal solutions within a reasonable amount of computation time.

***Index Terms*— Genetic algorithms, heuristic knowledge, resource-constrained project scheduling.**

I. Introduction

THE PROBLEM considered here involves a general category of the resource constrained project scheduling problem in which activities may have more than one execution mode and renewable as well as nonrenewable resource constraints exist. The amount of nonrenewable resource that can be consumed during the project is limited. Different activity execution modes require different amounts or different types of resources and represent alternative ways of realizing an activity. Each activity mode thus specifies a unique activity duration and a set of requirements for both renewable and nonrenewable resources. This problem is relevant in flexible manufacturing systems and other scheduling environments where alternate routings are permitted for jobs and different machines have different machining characteristics and tool and material handling requirements. The aim here is to minimize the makespan, or, in other words, the project duration. There are two decisions in determining the schedule with the optimum project duration: the selection of activity modes and the selection of the activity sequence, which respects renewable and nonrenewable resource constraints. The mathematical formulation for this problem is given in Talbot [38] and further extended in Patterson *et al.* [30]. Optimization approaches proposed to solve this problem are basically enumeration schemes [38], [30], [35]. These methods are not expected to find an optimal solution within a reasonable computation time when faced with large size problems since the considered problem is NP complete [4].

Heuristic solution approaches are discussed in a recent survey by Kolisch and Drexl [21], in which extensive experimentation is carried out on a set of 536 test problems [22] and four different heuristic methods [5], [38], [12], [21] are compared with the optimal solutions. Talbot's [38] exact method is used as a heuristic in terms of a truncated enumeration technique that results in a serial scheduling method. Boctor [5] proposes a parallel scheduling method in which activities are selected by the minimum total slack time (MINSK) rule in their shortest resource-feasible mode. Drexl and Grünewald [12] propose a biased random sampling approach that uses a serial scheduling method and the shortest processing time (SPT) rule. Kolisch and Drexl [21] provide a local search method controlled by a number of neighbor-moves deciding on activity modes first and then on the activity sequence. Among the heuristics that are not tested by Kolisch and Drexl [21] are two different procedures given by Boctor [6], [7] and the procedure given by Özdamar and Ulusoy [27]. We remark that, among the four tested methods, only the Kolisch/Drexl local search method finds feasible solutions for all 536 test instances and provides the least average deviation from the optimum project duration.

Here we propose a genetic algorithm (GA) approach to deal with this problem. GA's were first developed by Holland [19] and De Jong [10] and are based on the mechanics of natural selection in biological systems. GA's use a structured but randomized way to utilize genetic information in finding new directions of search. The major genetic operators that reflect nature's evolutionary process are reproduction, crossover, and mutation. Many variations of these operators along with their performance records may be found in the literature. An extended survey on GA's and their extensions is given in Michalewicz [25]. GA's are implemented in a wide variety of application areas, specifically in combinatorial problems, such as scheduling (see, for instance, [3], [9], [14], [26], [17], [18], [11], [2], [23], [24], [31], [8], [37], [20]). The role of GA's in combinatorial optimization problems is discussed by Goldberg [15], and the advantages of GA's over other search methods are summarized as follows.

Manuscript received October 7, 1995; revised October 31, 1996 and December 7, 1997.

The author is with the Department of Computer Engineering, Istanbul Kültür University, 80280 Istanbul, Turkey (e-mail: lozdamar@hotmail.com).

Publisher Item Identifier S 1094-6977(99)00101-7.

1094–6977/99$10.00 © 1999 IEEE

Authorized licensed use limited to: UNIVERSIDAD CARLOS III MADRID. Downloaded on June 29,2025 at 01:18:44 UTC from IEEE Xplore. Restrictions apply.

1) Search is carried out from a population of points, not a single point.
2) Payoff information is used instead of derivatives.
3) Probabilistic transition rules are used instead of deterministic ones.

An excellent survey on GA's in the area of scheduling is provided by Portmann [32], in which different GA designs integrated with other search techniques are discussed. Portmann [32] also analyses the effects of various GA characteristics, such as the encoding, crossover, and mutation operators on the performance of GA's in scheduling.

We adopt the "indirect chromosome representation" [36], [39] in our own GA approach; i.e., chromosomes are encoded by a series of priority rules that are applied by a schedule generator to build a solution. Thus, simple crossover and mutation operators can be applied to generate offspring without any repair requirements for preserving feasibility. The priority rules and the schedule generator are designed specifically to deal with the resource constrained scheduling problem, and, hence, problem-specific knowledge is incorporated into the GA approach. The GA approach proposed here is denoted as hybrid GA (HGA), and it integrates the concepts of natural selection and adaptation with existing scheduling methodology, which is successfully exploited for the problem under question. The benefits of HGA are demonstrated by comparing HGA with a pure GA (PGA), which does not integrate problem-specific knowledge.

II. HGA AND PGA

The HGA and PGA proposed here execute the reproduction, crossover, and mutation operations in each generation, after constructing the initial population. The algorithms run for a prespecified number of generations and convey the best objective function value, namely, the minimal project duration, obtained up to that generation. In the following, the elements of both HGA and PGA are discussed.

A. Encoding and Schedule Generator for HGA

1) Priority Rule-Based Encoding for HGA: Due to the complexity of the problem, a problem-specific encoding is necessary for simplifying crossover and mutation operators as well as utilizing valuable scheduling knowledge to achieve a good performance. We represent each individual by a number of genes equal to twice the number of activities in the project N. On each locus there exist two genes. The first gene specifies the execution mode m_n in which the nth activity is to be realized. The second gene specifies the priority rule Z_n according to which the candidate activity to be scheduled in the nth position is selected. In other words, during the construction of a schedule in a parallel scheduling algorithm, N activities are selected for scheduling over all decision time points. Consequently, the second gene identifies the manner in which the selection is to be carried out at the nth decision time. Herrmann *et al.* [18] use similar reasoning in dealing with a nonclassical job-shop with sequence-dependent setup times and multiple conflicting objectives. However, the representation in that reference is such that a gene specifies the priority rule according to which the sequence on the nth machine is to be constructed. Precedence feasibility is then provided, and a complete schedule is created by a scheduling system. Dorndorf and Pesch [11] also use a priority rule-based encoding for solving the classical job-shop scheduling problem to schedule candidate operations while generating active schedules.

Our representation is illustrated as follows through a project whose related information is given in Table I. Each activity has up to three modes with their respective durations and renewable/nonrenewable resource requirements. Nine units of the first and four units of the second renewable resource exist each period. At most, 29 and 40 units of the first and second nonrenewable resource types are allowed to be consumed over the whole project. In Table I, each activity's immediate predecessors are given. The first and last activities are dummy activities.

A valid chromosome for this project would be given as

Mode Assignment:	(1, 1, 3, 1, 1, 2, 2, 1, 3, 1, 3, 1)
Priority Assignment:	(1, 3, 2, 5, 9, 6, 2, 7, 7, 9, 6, 8)

where nine priority rules are specified. The nine priority rules used in this study are selected based on reported results in previous articles [1], [40], [41], and they are given in the Appendix.

These priority rules are among the best reported priority rules in the literature. Thus, the encoding subsumes all of the best domain knowledge about the problem while enjoying the flexibility of integrating multiple decision rules into a single scheduling scheme. Usually, a problem is solved with one of these rules, and sometimes, the treatment of all activities according to the same criterion throughout the whole schedule construction process puts the priority rule at a disadvantage due its restricted myopic vision. Here, the encoding is enriched with the support of more than one active priority rules in solving the problem.

The first set of genes represent the modes assigned to activities $1 \ldots N$, where N is the number of activities in the project. The mode assignment in the example chromosome represents a feasible mode assignment with respect to both nonrenewable resources; i.e., 15 and 38 units of nonrenewable resources are consumed for the two types, respectively. The second set of genes should read as follows: the first scheduling decision should be carried out by sorting schedulable activities according to the MINSK rule (priority rule #1) and selecting the highest priority activity; the second scheduling decision should be directed by the SPT rule (priority rule #3); and so on.

2) Schedule Generator for HGA: Each time a new offspring or a mutated chromosome is generated, the schedule generator is used to build the corresponding feasible schedule. The schedule generator consists of an iterative parallel scheduling algorithm [28], [29], [42] that performs exactly two iterations (in all of these references, more than two scheduling passes are permitted), one forward and one backward. The pseudocode for the schedule generator is given in Table II.

In the forward scheduling pass, activity priorities are calculated according to different priority rules, depending on

Authorized licensed use limited to: UNIVERSIDAD CARLOS III MADRID. Downloaded on June 29,2025 at 01:18:44 UTC from IEEE Xplore. Restrictions apply.

TABLE I
INFORMATION FOR THE EXAMPLE PROJECT

Activity	Mode	Duration	Renewable Resource(1)	Renewable Resource(2)	Nonrenewable Resource(1)	Nonrenewable Resource(2)	Predecessor Activities
1	1	0	0	0	0	0	-
2	1	3	6	0	9	0	1
	2	9	5	0	0	8	
3	1	1	0	4	0	8	1
	2	1	7	0	0	8	
	3	5	0	4	0	5	
4	1	5	7	0	2	0	1
	2	8	6	0	0	7	
5	1	6	2	0	0	7	2
6	1	2	2	0	8	0	2
	2	6	2	0	0	1	
7	1	3	5	0	10	0	5
	2	8	5	0	0	10	
8	1	4	6	0	0	1	5
	2	10	3	0	10	0	
9	1	2	2	0	6	0	4, 7, 8
	2	7	1	0	0	8	
	3	10	1	0	0	7	
10	1	1	4	0	4	0	3, 6, 7
	2	1	0	2	0	8	
	3	9	4	0	0	5	
11	1	6	0	2	0	10	3, 6
	2	9	0	1	0	9	
	3	10	0	1	0	7	
12	1	0	0	0	0	0	9, 11

activity early/late start/finish times determined by the critical path method (CPM). CPM uses the duration of the modes assigned to activities in the first set of genes of the chromosome. The scheduling time is initiated at time zero, and the set of schedulable activities (ELIGSET) is identified. A schedulable activity has all its predecessors completed, and the renewable resource requirements of its execution mode (specified by the first set of genes in the chromosome) do not exceed available levels at the current scheduling time. (The schedule generator does not need to consider the feasibility of the schedule with respect to nonrenewable resources since the first set of genes always specifies a feasible mode assignment. The latter condition is imposed both in the generation of the initial population and in the crossover and mutation operators.)

Next, priority values are determined for all activities in ELIGSET by using the incumbent priority rule Z_n in the chromosome. For instance, if, in the example project described in Table I, four activities have been scheduled up to the current scheduling time, all schedulable activities are sorted according to the fifth priority rule (fifth gene in the second set of genes of the example chromosome), $Z_5 = 9$ [most total successors (MTS)]. Each time an activity is scheduled, the position index n for the sequence of priority rules in the second part of the chromosome is incremented. Thus, for each scheduling decision, there exists a (possibly) different scheduling criterion (priority rule). After every scheduling action, the available renewable resource levels are decreased and activity status is updated. Then, ELIGSET is reidentified at the current scheduling time with respect to the updated resource levels and some other activity is scheduled. This step is repeated until ELIGSET becomes empty at the current scheduling time. Then, the scheduling time is advanced to the earliest completion time among activities in progress, resources released by completed activities are added to available levels, the activity status for completed activities is updated, and the scheduling step is repeated. The forward scheduling pass is completed when all activities are scheduled. Then the backward scheduling pass commences.

Authorized licensed use limited to: UNIVERSIDAD CARLOS III MADRID. Downloaded on June 29,2025 at 01:18:44 UTC from IEEE Xplore. Restrictions apply.

TABLE II
PSEUDOCODE FOR THE SCHEDULE GENERATOR FOR HGA

```
Procedure Generate Schedule for HGA (Offspring h /mutated chromosome h);
    begin
      Read genes of offspring h / mutated chromosome h;
      CPMSolve;
      Forwards = true; time = 0;
      Repeat
        if Backwards then
          for j = 1 to N do
            begin
              lft_j = makespan-statime_j
              est_j = makespan-fintime_j
            end;
        n = 1; {n = N in backward pass }
        Repeat
          Repeat
            Find ELIGSET;
            FindHighestPriorityActivity(j*) according to Z_n;
            Schedule(j*);
            n = n+1; {n = n-1 in backward pass}
            ELIGSET = ELIGSET- [j*];
          Until ELIGSET = ∅
          Time = min {C_k} ∀ k ε set of activities in progress;
          AdjustResources; AdjustActivityStatus;
        Until AllActivitiesAreScheduled;
        Backwards = true;
      Until Forward/BackwardIterationsAreCompleted;
    end; {Procedure}
```

Definitions:

$statime_j$: the scheduled start time of activity j in the forward schedule;
$fintime_j$: the scheduled completion time of activity j in the forward schedule;
makespan: the project duration obtained in the forward scheduling iteration;

Procedures:

CPMSolve finds lft,est,eft,lft values considering the durations of activity modes assigned in the chromosome;
FindHighestPriorityActivity sorts the activities in ELIGSET according to descending priority value of Z_n and selects the one at the top of the list, j*;
Schedule assigns activity j* at the current time and decreases the level of renewable resources accordingly and changes the status of activity j*;
AdjustResources adds the resource units released by the activities then completed;
AdjustActivityStatus adds the activities then completed to the completed activity set;
C_k is the completion time of activity k.

Backward scheduling is different from forward scheduling in one respect: in backward scheduling, activities become schedulable when their successors are completed. Otherwise, backward scheduling is exactly the same as forward scheduling and the initial scheduling time is also zero. In backward scheduling, the sequence of the second set of genes (priority rules) in the chromosome is used starting from the last one because the last activity of the project is the first activity to be scheduled. Each time an activity is scheduled, the position index n for the sequence of priority rules is decreased. Furthermore, before the backward scheduling pass, all activity time windows are revised and they become equal to the difference between the project duration obtained in the forward pass and their corresponding start (completion) times in the forward schedule (activity late finish time = forward schedule's makespan—activity start time in forward schedule; activity early start time = forward schedule's makespan—activity completion time in forward schedule).

Thus, activity priorities are automatically changed from their initial values before the backward pass begins since priority rules use information based on activity time windows. Activities that are scheduled early in the forward pass (due to the fact that they have tight initial time windows) have more relaxed activity late finish times in the backward pass and vice versa. The backward scheduling approach tends to right-shift the scheduled times of activities and, hence, relax early resource bottlenecks that occur in nondelay forward scheduling, and dynamic activity time windows raise the

Authorized licensed use limited to: UNIVERSIDAD CARLOS III MADRID. Downloaded on June 29,2025 at 01:18:44 UTC from IEEE Xplore. Restrictions apply.

TABLE III
SCHEDULE RESULTING FROM FORWARD SCHEDULING PASS

Initial Activity Time Windows

Activity	1	2	3	4	5	6	7	8	9	10	11	12
est	0	0	0	0	3	3	9	9	17	17	9	27
lft	0	3	17	17	9	17	17	17	27	27	27	27

Forward Pass

Time	ELIGSET	Position Index, n (priority sequence)	Z_n	Selected activity, j*	Activity Completion Time
0	# 1	1	1 (MINSK)	# 1	0
0	# 2,3,4	2	3 (SPT)	# 2	3
	# 3	3	2 (LFT)	# 3	5
3	# 4,5,6	4	5 (WRUP)	# 5	9
	# 4,6	5	9 (MTS)	# 6	9
5	∅				
9	# 4,7,8,11	6	6 (LST)	# 7	17
	# 11	7	2 (LFT)	# 11	19
17	# 4,8,10	8	7 (EST)	# 4	22
19	∅				
22	# 8,10	9	7 (EST)	# 8	26
26	# 9,10	10	9 (MTS)	# 9(arbitrary)	36
	# 10	11	6 (LST)	# 10	27
36	# 12	12	8 (EFT)	# 12	36

possibility of constructing a totally different schedule for the same chromosome. Via forward–backward scheduling, the schedule generator for HGA introduces the very basic idea of shuffling resource requirements to smooth out irregularities in the resource consumption levels over the project span. Thus, knowledge related to problem domain is also used in the schedule generator.

3) Example Schedule Generated by the Schedule Generator for HGA: The mechanics of the schedule generator and the implications of a priority rule-based chromosome are best demonstrated through the example chromosome given for the problem instance in Table I. (The precedence weight w_p used in priority rule #5 is assumed to be 0.5 for this example.) The steps taken by the schedule generator during the construction of the forward and backward schedules are indicated in Tables III and IV, respectively. In Tables III and IV, the activity time windows valid before the two scheduling passes are executed are also indicated at the top of both tables. The Gantt charts obtained in the forward and backward passes are depicted in Figs. 1 and 2. In Figs. 1 and 2, the placement of each activity within its permitted time window is demonstrated.

Let us explain how the schedule generator builds the schedule by utilizing the first string genes in the chromosome (the sequence of priority rules). At time zero, after the first dummy activity is scheduled, the second, third, and fourth activities become schedulable. The position index n for the sequence of priority rules is equal to two (the first position is allocated to the dummy activity). Since the second gene in the priority rule string is three (SPT rule), the second activity, which has the shortest duration (in its first mode) among all schedulable activities, is selected for scheduling at time zero. The position index then becomes three. Next, since three and four units of the two renewable resources are now available and the fourth activity requires seven and zero units of resources, it is not included in ELIGSET, which is reidentified every time an activity is scheduled. ELIGSET now consists of the third activity (in its third mode), which is also scheduled at time zero. (It does not matter which priority rule is indicated in the third position of the genes representing priority rules because there is a single activity to be scheduled.) Then, the available renewable resource levels then drop to three and zero units for the two types, and the reidentified ELIGSET becomes empty. Next, the time is advanced to three, which is the earliest completion time among activities in progress (the second and third activities are in progress). The status of the second activity is changed, and the available resource levels become nine and zero units for the two resource types. The scheduling process continues with the next priority rule (#5) indicated in the fourth position in the sequence of genes.

In Table IV, the revised activity time windows are indicated as determined by the activity scheduled times in the forward

Authorized licensed use limited to: UNIVERSIDAD CARLOS III MADRID. Downloaded on June 29,2025 at 01:18:44 UTC from IEEE Xplore. Restrictions apply.

TABLE IV
SCHEDULE RESULTING FROM THE BACKWARD SCHEDULING PASS

Revised Activity Time Windows (activity scheduled times in forward schedule)												
Activity	1	2	3	4	5	6	7	8	9	10	11	12
est	0	0	0	17	3	3	9	22	26	26	9	36
lft	0	3	5	22	9	9	17	26	36	27	19	36
Revised Activity Time Windows (Reversed for backward pass)												
est	36	33	31	14	27	27	19	10	0	9	17	0
lft	36	36	36	19	33	33	27	14	10	10	27	0

Backward Pass

Time	ELIGSET	Position Index, n (priority sequence)	Z_n	Selected activity, j*	Activity Completion Time
0	# 12	1	8 (EFT)	# 12	0
0	# 9,10,11	2	6 (LST)	# 9	10
	# 10,11	3	9 (MTS)	# 10	1
	# 11	4	7 (EST)	# 11	10
1	∅				
10	# 3,4,6,7,8	5	7 (EST)	# 8	14
	# 3,6	6	2 (LFT)	# 6	16
	# 3	7	6 (LST)	# 3	15
14	# 4,7	8	9 (MTS)	# 7	22
15	∅				
16	∅				
22	# 4,5	9	5 (WRUP)	# 4	27
	# 5	10	2 (LFT)	# 5	28
27	∅				
28	# 2	11	3 (SPT)	# 2	31
31	# 1	12	1 (MINSK)	# 1	31

scheduling pass. Furthermore, they are reversed; i.e., activity late finish (early start) times (LFT) become equal to the difference between the forward schedule's makespan and activity scheduled start (completion) times, so that they become meaningful for the backward pass, which schedules the last activity of the project at time zero. Notice that the sequence provided by the LFT rule, for example, determined by the initial activity LFT's, is different from the one provided by the revised time windows (e.g., the initial LFT's of the third and fourth activities are equal, but the revised values are different). Namely, the priority values of activities have changed as compared to the forward scheduling pass.

In Fig. 2, the Gantt chart of the schedule obtained by the backward pass is demonstrated. (Activity scheduled times and the revised activity time windows in Fig. 2 are adapted to a forward scheduling perspective.) When the Gantt charts in Figs. 1 and 2 are compared, the change in the resource-critical activity sequences constructed by the forward and backward scheduling passes become obvious. The latter sequence is composed of the string of activities 2-7-4-8 in the forward schedule and 2-4-7-8 in the backward schedule. Activity 6 is interchanged with activity 4 in the backward pass (activities 4–6 constitute a local resource conflicting subset at time period three in the nondelay forward pass) and activity 8 can thus be scheduled comparatively earlier to result in an improved makespan of 31 periods. (A local resource conflicting subset is a set of activities whose resource requirements exceed the available resource levels at the incumbent scheduling time.) The iterative scheduling algorithm has two effects on the schedule corresponding to a chromosome. First, through dynamic time, windows activity priorities are rehashed. Second, the schedule is right-shifted by backward scheduling; e.g., activities 3, 10 and 11 are pulled to the right in the backward pass. Although right-shifting these activities does not improve the makespan in this example, in other test instances, it might have the effect of dissolving the conflicts in some of the local resource conflicting subsets whose members are forced to overlap in the forward scheduling pass as well as creating new local resource conflicting subsets. When combined with revised activity time windows and revised

Authorized licensed use limited to: UNIVERSIDAD CARLOS III MADRID. Downloaded on June 29,2025 at 01:18:44 UTC from IEEE Xplore. Restrictions apply.

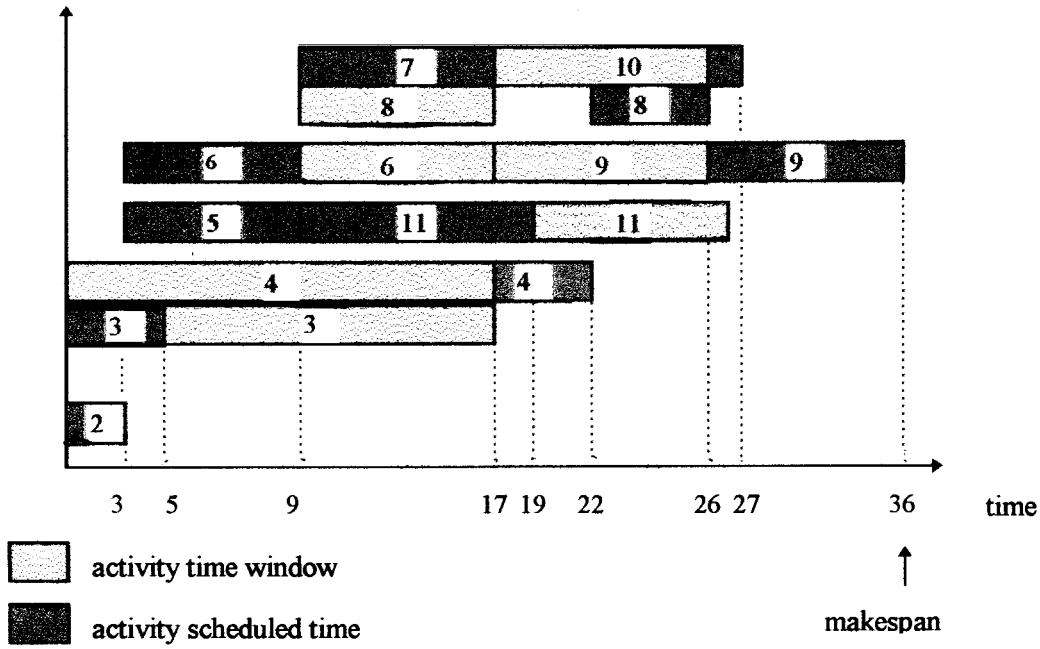

Fig. 1. Activity time windows and scheduled times in the forward scheduling pass.

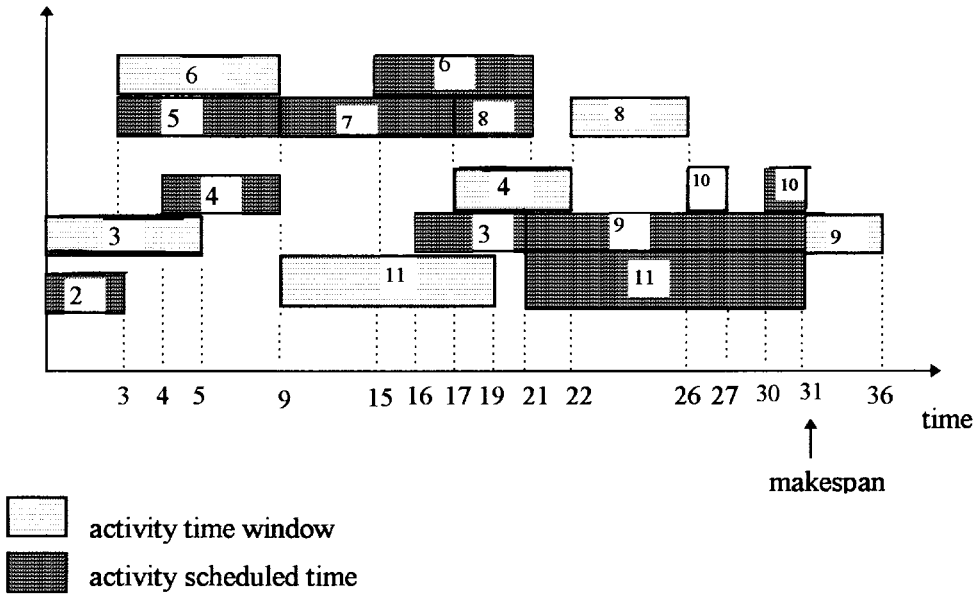

Fig. 2. Activity time windows and activity scheduled times in the backward scheduling pass (adapted to a forward scheduling perspective).

activity priority values, right-shifting activities contributes to constructing different local resource conflicting subsets and different resource-critical activity chains. Consequently, a joint effect of rearranging the activity sequence takes place in the backward scheduling process. Note that the iterative scheduling algorithm described here and the benefits obtained from the joint effect are applicable when the priority rule-based encoding of HGA is used. Thus, we emphasize the flexibility of the encoding developed for HGA in terms of the different and possibly improved activity sequences that can be constructed for the same chromosome.

B. Encoding and Schedule Generator for PGA

1) Sequence-Based Encoding for PGA: In PGA, an encoding that does not utilize problem-specific knowledge is utilized. The encoding consists of a set of mode assignment genes and an ordered set of genes that describe an "activity sequence." The activity sequence strictly respects the precedence relationships among activities. Unfortunately, the sequence-based encoding does not have a one-to-one mapping to a solution in the problem under consideration because the sequence of the resource-critical activities determines the makespan and the other activities may float within their time windows without

Authorized licensed use limited to: UNIVERSIDAD CARLOS III MADRID. Downloaded on June 29,2025 at 01:18:44 UTC from IEEE Xplore. Restrictions apply.

TABLE V
PSEUDOCODE FOR THE SCHEDULE GENERATOR FOR PGA

```
Procedure Generate Schedule for PGA (Offspring h /mutated chromosome h);
    begin
        Read genes of offspring h / mutated chromosome h;
        time = 0; n = 1;
        Repeat
            j* = A_n;
            If j* is schedulable then
                begin
                    Schedule (j*);
                    n = n+1;
                end
            Else begin
                    Time = min {C_k} ∀ k ε set of activities in progress;
                    AdjustResources; AdjustActivityStatus;
                end;
        Until AllActivitiesAreScheduled;
    end; {Procedure}
```

Definition:

A_n = activity in the nth position in the sequence specified by the chromosome h.

affecting the makespan. The schedule observed in Fig. 1 could have been described by the ordered set of activities (1–3, 5–7, 11, 4, 8–10, 12) or by (1, 3, 2, 5–7, 11, 4, 8–10, 12), or by other sequences that preserve the resource-critical activity sequence (2-7-4-8) and end up in the same makespan. It is difficult to eliminate the latter issue by encoding the resource-critical activity chain only because, for each different set of mode assignment genes (there exist an exponential number of different mode assignment sets), we need to find out the resource-critical activity sequence, which requires the evaluation of all possible resource conflicting subsets of activities that overlap in their time windows. However, the sequence-based encoding proposed here is pure in the sense that it represents the problem without referring to a heuristic space. This type of encoding requires no effort to conceive and is more general in the sense that it can be applied to many sequencing problems without modification. On the other hand, HGA requires additional effort to build the expert knowledge and therefore has the potential to achieve good quality results quite fast.

2) Schedule Generator for PGA: The schedule generator for HGA is modified to construct one of the schedules consistent with the resource-critical activity chain in the sequence-based encoding devised for PGA. The pseudocode of the schedule Generator for PGA is found in Table V. Here activities are scheduled in the order of the encoded sequence at their earliest possible times with respect to precedence and resource constraints. In the modified schedule generator, at every scheduling decision time, if the nth encoded activity in the sequence A_n is schedulable at the current scheduling time, it is scheduled. Each time an activity is scheduled, the position index n of the sequence is incremented. Otherwise, the schedule generator waits for A_n to become schedulable and advances the scheduling decision time without attempting to schedule other activities that are schedulable then. Namely, idle time is inserted into the schedule and there is no guarantee that a nondelay schedule will result. On the other hand, there is also a possibility of constructing an active schedule that is not nondelay. The latter would be an advantage if the optimum solution lies in the active schedule region only. In Fig. 3, we demonstrate the schedule resulting from the sequence-based chromosome (1, 2, 5, 4, 7, 3, 6, 8, 11, 10, 9, 12) that implies the resource-critical sequence (2-4-7-8). Note that, at period zero, activity 3 is also schedulable, but it is the sixth activity in the sequence and only two activities (activities 1 and 2) have been scheduled. So, activity 3 has to wait until the ninth period, when activity 7, the fifth activity in the sequence, is scheduled. The schedule in Fig. 3 is semiactive.

Unlike the priority rule-based encoding, the sequence-based encoding is rigid in the sense that the sequence is independent of the activity time windows and backward scheduling cannot improve the makespan by right-shifting activities since the sequence is fixed. Thus, the backward pass schedule for this chromosome would result in a Gantt chart exactly the same as

Authorized licensed use limited to: UNIVERSIDAD CARLOS III MADRID. Downloaded on June 29,2025 at 01:18:44 UTC from IEEE Xplore. Restrictions apply.

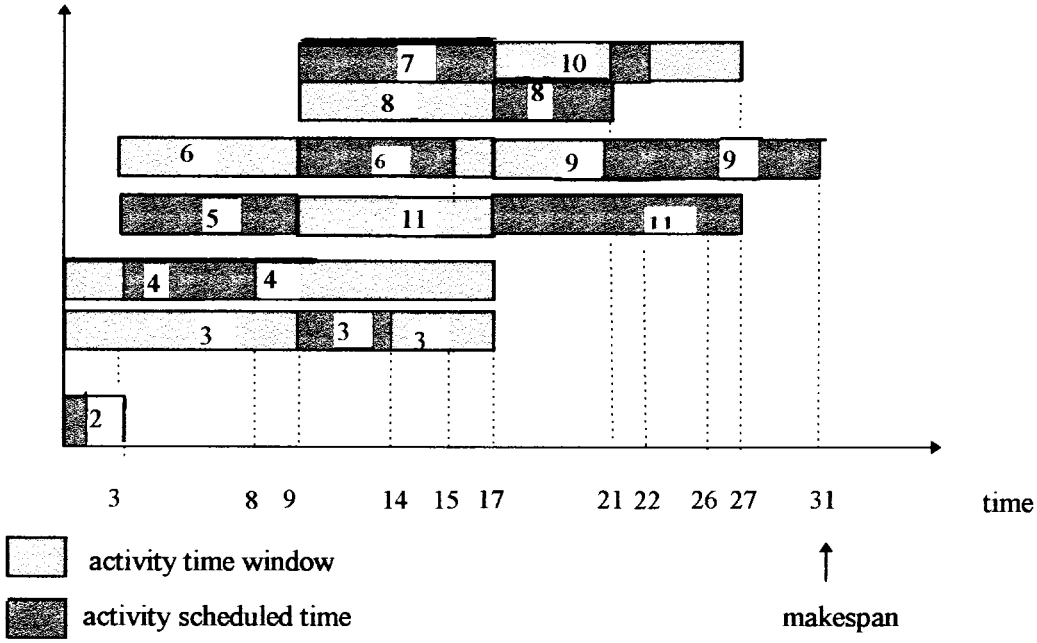

Fig. 3. Activity time windows and scheduled times in the forward scheduling pass (PGA, sequence-based encoding).

the one found in Fig. 2. Such an encoding cannot benefit from the joint effect of the backward scheduling pass and dynamic activity time windows.

C. Reproduction

The reproduction strategy used in both HGA and PGA is a pure selection procedure [16]. Each chromosome in the population is reproduced a number of times proportional to its objective function value. The fitness function used for selecting a chromosome is a polynomial of the inverse of the chromosome's makespan, produr_k. A polynomial is preferred over a linear fitness function to obtain a faster convergence. The selection probability for a chromosome k is given as

$$\text{prob}_k = \frac{(1/\text{produr}_k)^2}{\sum_{k\varepsilon\,\text{population}}(1/\text{produr}_k)^2}.$$

The cumulative probabilities for generating different selection intervals are given as

$$\text{cum}_j = \sum_{k=1}^{j} \text{prob}_k, \qquad \text{for } j = 1 \ldots \text{popsize}.$$

Here, popsize is the population size. Reproduction is carried out by calculating first the cumulative, cum_j for all j, to generate (popsize-1) number of intervals. Then, a random number r, less than or equal to one, is generated. If $\text{cum}_j \leq r < \text{cum}_{j+1}$, the chromosome j is selected as a parent. Reproduction is repeated exactly popsize times to generate the parent population.

D. Crossover

1) Two-Point Crossover for HGA: A simple two-point crossover is used for HGA, in which two random numbers $r1$ and $r2$ specify the length of the string that is to be swapped to create offspring. Since the second part of the chromosome involves a string of priority rules, there are no repair requirements for preserving precedence and resource feasibility in the offspring. However, infeasible offspring may result with respect to nonrenewable resources in the mode assignment part of the chromosome because both sets of genes are swapped during the crossover operation. For instance, the following two parents:

Parent 1: Mode Assignment:
(1, 1, 3, 1, 1, 2, 2, 1, 3, 1, 3, 1)
Priority Assignment:
(1, 3, 2, 5, 9, 6, 2, 7, 7, 9, 6, 8)

Parent 2: Mode Assignment:
(1, 2, 3, 1, 1, 2, 1, 2, 1, 3, 3, 1)
Priority Assignment:
(2, 6, 8, 5, 4, 7, 9, 3, 2, 8, 1, 6)

result in the following offspring given that $r1 = 2$ and $r2 = 7$:

Offspring 1: Mode Assignment:
(1, 1, 3, 1, 1, 2, 1, 1, 3, 1, 3, 1)
Priority Assignment:
(1, 3, 8, 5, 4, 7, 9, 7, 7, 9, 6, 8)

Offspring 2: Mode Assignment:
(1, 2, 3, 1, 1, 2, 2, 2, 1, 3, 3, 1)
Priority Assignment:
(2, 6, 2, 5, 9, 6, 2, 3, 2, 8, 1, 6).

Notice that the first offspring is resource feasible with respect to both nonrenewable resources, whereas the second offspring's mode assignment is not feasible with respect to the second nonrenewable resource type. However, the second offspring is not evaluated since there exists a control for resource feasibility before any evaluation is carried out.

Authorized licensed use limited to: UNIVERSIDAD CARLOS III MADRID. Downloaded on June 29,2025 at 01:18:44 UTC from IEEE Xplore. Restrictions apply.

2) Two-Point Crossover for PGA: For the sequence-based encoding in PGA, we apply the two-point linear order crossover (LOX) described in Falkenauer and Bouffouix [13] designed for sequencing problems because no elements are to be repeated in a sequence-based chromosome. LOX is described in detail in the latter reference, and it has the characteristic of preserving the relative positions between the genes. An example is given as follows: two parent chromosomes of five activities, parent 1: (3, |2, 4| 1, 5) and parent 2: (1, |4, 5| 2, 3), are given with indicated cross sections to be swapped. If plugged into parent 1, the cross section of parent 2 would cause a repetition of activity 5 in the sequence. So activity 5 is removed from parent 1 and activity 2 that does not overlap with the cross section of parent 2 is slid into position 4, while activity 1 is slid into position 5. Thus, the first child would be (3, 4, 5, 2, 1), and the second child (1, 2, 4, 5, 3) is obtained in the same manner. A repair procedure is added to LOX to preserve the precedence constraints. For each activity k in position i in the offspring, the repair step checks if any activities placed after position i are logical predecessors of activity k. If such a precedence violation is caused by an activity v in position $j(j > i)$, activity k is removed from position i and placed in position j and the partial sequence between i and j, including position j, is pushed one position to the left. Thus, activity v takes its place in position $j-1$ and the sequence is not disrupted much.

3) Uniform Crossover for HGA: As an alternative to the simple two-point crossover operator, we also test in preliminary runs for HGA the uniform crossover [43], which is reported to be superior to other crossover operators by Syswerda [37]. In uniform crossover, positions take up the values of one parent randomly and the empty positions take up the unused values in the order of the other parent. We do not consider the uniform crossover for PGA because the repair step would be considerably more burdensome computationally.

In both HGA's and PGA's, two-point crossover operators, only resource feasible recombinations of modes, are permitted and the parent population is totally replaced by the offspring population. The same principles are applied in uniform crossover.

4) Adaptive Crossover Rates: The crossover probability for a chromosome k is calculated as follows:

$$\mathrm{pc}_k = \frac{\mathrm{maxprodur} - \mathrm{produr}_k}{\mathrm{maxprodur} - \mathrm{minprodur}} \quad \text{if minprodur} \neq \text{maxprodur} \qquad (1)$$
$$\mathrm{pc}_k = 1.0, \quad \text{otherwise}$$

where minprodur is the least (best) makespan in the current population and maxprodur is the worst one. A chromosome's crossover probability is dependent on both its own makespan and the population's makespan range. The lower a chromosome's makespan, the higher is its probability for crossover, so that "good" genes get a higher opportunity to recombine in the next generation. Furthermore, a chromosome's crossover probability keeps changing according to the convergence of the population's makespan range adapting itself to every generation's performance. The crossover probability increases when the population's makespan range tends to get stuck at a local optimum. The crossover probability defined here is a variation of the adaptive crossover probability described by Srinivas and Patnaik [34].

5) Fixed Crossover Rates: Although Srinivas and Patnaik [34] report that adaptive crossover and mutation probabilities have a superior performance in the optimization of multimodal functions, we are not sure whether the adaptive crossover approach will work well for the problem considered here. Therefore, we compare the adaptive crossover approach with a fixed crossover rate between the values 0.75 and 0.95. The interval is found to be the best performing by Schaffer *et al.* [33] after extensive research.

E. Mutation

1) Mutation Operator for HGA: Mutation for HGA simply involves a random modification on a selected pair of mode assignment and priority rule genes. The mutated mode and priority rule need not occupy the same locus. Mutation is permitted only if resource feasibility with respect to nonrenewable resources is preserved.

2) Mutation Operator for PGA: Mutation in PGA involves a pairwise swap of two activities in the sequence. Precedence-infeasible activity swaps are not permitted. Mutation on mode assignment genes is similar to the one described for HGA.

3) Adaptive Mutation Rates: The mutation probability pm_k is calculated in the same manner as the crossover probability, namely, as in (1). However, mutation is only applied to a single randomly selected locus in the whole chromosome. Consequently, although the mutation rate is seemingly of the same magnitude as the crossover rate, the mutation rate is reduced in proportion to the number of activities in the project. So, the mutation rate for each gene in the chromosome is actually pm_k/N, where N is the number of activities in the project.

4) Fixed Mutation Rates: We also try the fixed mutation rate against the adaptive mutation rate. The fixed mutation rate assumes a magnitude in the interval 0.005–0.01 [33].

III. Testing Environment

The test problems used to evaluate HGA are the 536 problems used by Kolisch and Drexl [21] as a comparison basis. Each problem has ten activities with three modes per activity. There exist two types of renewable and two types of nonrenewable resources. The specific characteristic that makes these problems more complex is the existence of more than one nonrenewable resource type. In the case in which the nonrenewable resource availability is tight, it is very hard to locate the feasible solution space.

The test problems represent a full factorial design based on two major factors, the resource strength (RS) and the resource factor (RF), as defined in Kolisch *et al.* [22]. RF represents resource requirements in binary form; i.e., if an activity requires a resource type, it is assigned a value of one. RF is the sum of all such assignments divided by the number of renewable (nonrenewable) resources multiplied by the number of modes and the number of activities, N. On the other hand, RS reflects the resource tightness of the earliest start time

Authorized licensed use limited to: UNIVERSIDAD CARLOS III MADRID. Downloaded on June 29,2025 at 01:18:44 UTC from IEEE Xplore. Restrictions apply.

TABLE VI
SUMMARY OF RESULTS REPORTED IN KOLISCH AND DREXL (1994)

	Boctor [5]	**Talbot [38]**	**Drexl and Grünewald [12]**	**Kolisch and Drexl [21] (Number of Neighbour-Moves)**			
				10	**50**	**100**	**500**
Problems for which a feasible solution is found (%)	51.86	57.83	80.00	100	100	100	100
Average Deviation from Optimum (%)	14.89	15.91	10.88	10.24	5.72	4.08	1.75
Standard Deviation (%)	18.05	18.59	17.07	13.65	9.38	7.68	4.87
Average CPU sec.	0.00	4.40	0.45	0.06	0.17	0.29	1.22

schedule; i.e., it demonstrates the amount of resource required by parallel activities.

For the full factorial design mentioned above, RS assumes the four levels: 0.2, 0.5, 0.7 and one, whereas RF assumes two levels: 0.5 and one. Although 640 test problems are reported to be generated with ten replicates for each level combination, only 536 of them proved to have a feasible solution. Optimal solutions to these problems are provided by Kolisch *et al.* [22]. The authors have used the Patterson *et al.* [30] procedure to find the optimal solution. It is noted by the authors that, as RF increases for both renewable and nonrenewable resources, the problems become more difficult to solve. However, the increase in RF for nonrenewable resources is far more effective than the one for renewable resources. Furthermore, if RS is at a low level for nonrenewable resources $(\mathrm{RS} = 0.2)$, the problems become very difficult to solve, whereas at the same level of RS for renewable resources, the problem difficulty diminishes. In other words, the problems tend to be more difficult with increasing (decreasing) levels of renewable (nonrenewable) resource availability.

In order to challenge HGA, we have generated 32 more test problems, each with 90 activities and two modes/activity. Two renewable and two nonrenewable resource types exist. Again, the problems represent a full factorial design with RF held at two levels (0.5, 1.0) and RS held at two levels (0.2, 1.0) for both renewable and nonrenewable resources. There are two replicates for each factor level combination. The problems are also created by using the same problem generator, PROGEN [22], used for generating the 536 small test problems. However, due to the size of the problems, it is not possible to find optimal solutions. Consequently, the results obtained by HGA are compared to the ones obtained by PGA and a random search procedure (RSP), which applies an equal number of functional evaluations as HGA and PGA. The RSP generates the first set of genes (activity modes) randomly, and then, if the mode combination is resource feasible, the procedure generate schedule is called upon by generating a random priority rule for each candidate activity and performing the forward/backward pass. Thus, RSP generates a prespecified number of chromosomes within the feasible region and evaluates them. RSP lacks the search lead of a GA and involves only scheduling expertise. In a sense, RSP is the counterpart of PGA, which lacks scheduling expertise but consists of GA search only.

IV. RESULTS

A. Results for 536 Problems

1) Results Obtained by Heuristics Reported by Kolisch and Drexl [21]: The results obtained by HGA are compared to the ones obtained in the survey by Kolisch and Drexl [21]. The Kolisch and Drexl survey results are listed in Table VI in terms of the percentage of problems for which a feasible solution is found, the average percentage deviation from the optimum, the standard deviation of the percentage deviation from the optimum, and the average CPU time. The problems tested in Kolisch and Drexl are the 536 test problems used here. The CPU times found in Kolisch and Drexl are reported to have been obtained on a 386dx40 PC, whereas the GA results are obtained on a Pentium 75 PC. Therefore, CPU times are not comparable.

In their paper, Kolisch and Drexl report that the MINSK rule is used in Boctor [5]. A time limit of 10 CPU s is imposed on the truncated branch and bound procedure given by Talbot [38], and a sample size of 100 is selected for the sampling approach of Drexl and Grünewald [12]. In Table VI, different numbers of neighbor-moves are reported for the Kolisch and Drexl [21] local search procedure along with their performance measure values. It is observed that the CPU times increase linearly with the number of moves, whereas the solution quality increases more rapidly.

B. Results Obtained by HGA Approach

1) Effects of System Parameters on HGA: In order to test the effects of parameters on HGA, a variety of results (indicated in Table VII) are obtained using different combinations of system parameters, such as the generation number (GN) at which the search is terminated; the popsize; and the number of different random seeds (NRS) that HGA is allowed to start

Authorized licensed use limited to: UNIVERSIDAD CARLOS III MADRID. Downloaded on June 29,2025 at 01:18:44 UTC from IEEE Xplore. Restrictions apply.

TABLE VII
RESULTS OBTAINED BY HGA USING DIFFERENT SYSTEM PARAMETERS AND COMPARISON WITH KOLISCH AND DREXL (1994) RESULTS WITH 500 NEIGHBOR-MOVES

HGA	GN = 20, NRS = 5, popsize =30	GN = 20, NRS = 10, popsize =30	GN = 20, NRS =10, popsize =50	GN = 20, NRS = 20, popsize =50	GN = 40, NRS = 10, popsize =30	GN = 40, NRS = 20, popsize =30	GN = 70, NRS = 10, popsize =30	KD
Average Deviation from Optimum (%)	0.86	0.38	0.15	0.11	0.29	0.09	0.34	1.75
Standard Deviation (%)	2.66	1.83	1.02	1.01	1.72	0.82	1.64	4.87
% of problems for which an optimal solution is found	88.05	94.40	97.57	98.69	96.26	98.50	94.77	n.a.
Average CPU sec	1.50	2.82	4.29	8.50	4.41	11.27	10.45	-

KD : Results of Kolisch and Drexl [21] with 500 neighbour-moves

from. When $\mathrm{NRS} = 10$, ten runs are carried out by HGA, each with a different initial seed.

Observing the preliminary results in Table VII and considering solution quality as well as computational efficiency, the preferred system parameters for HGA might be those of the second or third columns. As far as the system parameters are concerned, the effect of GN is rather insignificant when compared to that of NRS. This result is quite expected since the initial population is most likely to be affected by the initial random seed. A higher number of NRS would imply a wider variety of initial populations, thus increasing the chances for better performance. As for the popsize, it also has an effect on the quality of the solutions and on CPU times. A larger popsize of 50 chromosomes smooths out the CPU times of three very hard problems, which are otherwise (popsize $= 30$) solved in about 175 CPU s. When a popsize of 50 chromosomes is used, the maximum solution time observed is 13 CPU s. These problems have tightly constrained nonrenewable resources, and a larger population leads to a higher probability of finding resource-feasible offsprings during crossover. Popsizes lower than 30 do not give satisfactory results. For example, with popsize $= 10$, $\mathrm{GN} = 20$, and $\mathrm{NRS} = 10$, the average percentage deviation becomes $1.25\% \pm 3.37\%$ and the average solution time is 3.77 CPU s. Notice that, when popsize becomes really small, the CPU times increase due to the difficulty of finding feasible offspring.

Under all tested system parameters, HGA finds feasible solutions in all problems. Furthermore, HGA always finds an average percentage deviation below 1% and solves more than 90% of the test problems to optimality. The solution times are also reasonable for a GA. A point to emphasize is that the performance of HGA is quite robust, and it is not affected by problem characteristics, such as the tightness of resource constraints (the effects of factor combinations are statistically insignificant). However, for three particular problems with tight resource constraints, the CPU time is affected due to the effort spent on identifying feasible offspring, specifically when the popsize is relatively reduced.

2) Empirical Convergence of HGA: The initial population in HGA is constructed by randomly generating strings of feasible mode assignments and priority rules. On the average, the best solution belonging to the initial population deviates from the optimum by 9.66% ($\pm$8.65%). Comparing the latter percentage deviation with HGA's final average deviation of less than 0.40%, we note that HGA provides a robust and fast converging search method for the problem considered here.

Another indicator of empirical convergence is the number of generations within which the best solution is obtained. For instance, in the third experiment ($\mathrm{GN} = 20$, $\mathrm{NRS} = 10$, popsize $= 50$), the best solution is found within four generations on the average and the maximum number of generations within which the best solution is found is 14. We consider this convergence rate remarkable for a GA implementation.

3) Effects of the Crossover/Mutation Operators and Rates: The two-point crossover in HGA is replaced by uniform crossover under the system parameters $\mathrm{GN} = 20$, $\mathrm{NRS} = 10$, and popsize $= 50$. It is observed in Table VIII that there is no statistically significant difference between the performance of the two operator types. In Table VIII, the results of HGA are also observed when the two-point crossover operator and fixed crossover/mutation rates are combined. These fixed rates are selected as a consequence of various preliminary tests carried out on 100 randomly selected problems from among the 536 problem set. Analyzing the results in Table VIII, we can conclude that the adaptive crossover/mutation rates are significantly effective on the performance of HGA. Based on these results, there is no need to test the fixed crossover/mutation rates on PGA.

4) Contribution of Problem-Specific Knowledge to a GA: Comparison of HGA with PGA and RSP: To determine the contribution of scheduling knowledge to the GA approach, HGA is compared with PGA in Table IX. Both PGA and HGA have adaptive crossover/mutation rates, but different types of two-point crossover operators are applied, as described previously. The parameters used in the experiment are $\mathrm{GN} = 20$, $\mathrm{NRS} = 10$, and popsize $= 30$. However, since the schedule generator in PGA does not perform the backward pass, its population size is set to 60 so that it carries out the same number of functional evaluations as HGA. HGA is

Authorized licensed use limited to: UNIVERSIDAD CARLOS III MADRID. Downloaded on June 29,2025 at 01:18:44 UTC from IEEE Xplore. Restrictions apply.

TABLE VIII
EFFECTS OF DIFFERENT CROSSOVER OPERATORS AND CROSSOVER/MUTATION RATES ON HGA

HGA GN = 20, NRS = 10, popsize = 50	two-point crossover + adaptive crossover / mutation rate	uniform crossover + adaptive crossover / mutation rate	two-point crossover + fixed crossover rate = 0.95, mutation rate = 0.005
Average Deviation from Optimum (%)	0.15	0.28	3.44
Standard Deviation (%)	1.02	1.41	5.48
% of problems for which an optimal solution is found	97.57	96.26	60.44

TABLE IX
COMPARISON OF HGA, PGA, AND RSP ON 536 TEST PROBLEMS

	HGA	PGA (popsize = 60)	RSP
Average Deviation from Optimum (%)	0.38	3.88	1.88
Standard Deviation (%)	1.83	5.39	3.76
% of problems for which an optimal solution is found	94.40	59.88	75.00
Average CPU sec	2.82	2.37	2.12

(System Parameters: GN = 20, NRS = 10, popsize = 30)

also compared with the procedure RSP, which is a random walk involving only scheduling knowledge. The same number of functional evaluations are employed in RSP as HGA and PGA. In RSP, for each problem, the minimum makespan among 6000 randomly generated feasible chromosomes is compared with the optimal solution. Similar to HGA and PGA, RSP is allowed to start from a new random seed after every 600 functional evaluations. In Table IX, it is observed that RSP results in an average percentage deviation of 1.88% $\pm$ 3.76%, which is definitely better than PGA, and definitely worse than HGA. Although the average quality of RSP is nearly the same as the local search method given by Kolisch and Drexl [21], its performance in finding the optimal solution is poor.

PGA's computation time is relatively lower than HGA's due to the fact that its schedule generator does not require CPM calculations in every chromosome evaluation. The comparatively worse performance of PGA indicates that a GA scheme that does not incorporate relevant problem knowledge is not as successful as a GA scheme that does. HGA thoroughly exploits the support provided by the knowledge base involved in the scheduling rules, the advantages of the iterative schedule generator and the powerful GA search technique.

These results confirm the claim by Wolpert and Macready [44] in their article on the "No Free Lunch" theorem. The claim is that a general search algorithm (such as PGA in our context) is applicable to a wide range of scheduling problems, but it provides inferior results for specific problems, such as the one considered here. On the other hand, a search algorithm using domain-specific knowledge naturally outperforms the standard algorithm, but it cannot be used for other problems due to the difficulty of transferring expert knowledge that has to be revised for the new problem. Thus, there is a tradeoff between spending some time on developing the expertise required to solve the problem (and getting good results) and constructing (very fast) a simple search algorithm that will obtain inferior results.

5) Evaluation of HGA Against Existing Heuristics: The difficulty of the problem under consideration prevents simple constructive algorithms from finding feasible and optimal solutions. As demonstrated in the Kolisch and Drexl [21] survey, general search algorithms are more appropriate for this kind of combinatorial problem. Although HGA results are significantly superior to those obtained by earlier heuristics, in our opinion, the point to emphasize here is neither the amount of improvement in the performance measures nor the comparison of the computational times required by all heuristics. The aim in the development of HGA is to incorporate expertise into the GA approach and set a conceptual example for future GA implementations in solving complicated decision-making problems in which the inbred knowledge of the decision makers is indispensable.

Authorized licensed use limited to: UNIVERSIDAD CARLOS III MADRID. Downloaded on June 29,2025 at 01:18:44 UTC from IEEE Xplore. Restrictions apply.

TABLE X
RESULTS FOR 32 LARGE SIZE PROBLEMS

	HGA GN = 20,NRS = 1, popsize = 50	**PGA** GN = 20,NRS = 1, popsize = 100	**RSP** 1000 functional evaluations
Average Deviation from Best Solution (%)	0.85	19.51	7.12
Standard Deviation (%)	1.83	11.88	7.2
number of problems for which the overall best solution is found	28	0	8
Average CPUsec.	178	55	343

C. Results for the Second Set of Problems

Since for small problems good results are provided under the third column in [Table VII] in a reasonable solution time, the corresponding set of parameters ($\mathrm{GN} = 20$, $\mathrm{NRS} = 10$, popsize $= 50$) are selected for HGA and PGA in solving the large size problems. Popsize is doubled for PGA. However, $\mathrm{NRS} = 1$ in this case. Table X demonstrates the results for large size problems. Unfortunately, it is not possible to solve these problems to optimality due to their size. Consequently, instead of the average percentage deviation from the optimum criterion, the average percentage deviation from the overall best solution is utilized as a measure of solution quality. For RSP, 1000 functional evaluations are allowed. The number of best solutions is also counted for HGA, PGA, and RSP.

Looking over the results in Table X, we remark that the difference between HGA, PGA, and RSP with respect to the quality of solutions is far more marked in the case of large size problems. This is an expected result since, for large size problems, the feasible solution space is much larger and the probability of obtaining good solutions by random walk or by a pure GA encoding is lower. PGA's performance has deteriorated intolerably because the number of possible sequences increases and PGA is required to spend much more effort to achieve good results. However, in this experiment, the number of functional evaluations carried out by all three approaches is kept at the same level for a fair comparison.

The contradictory relationship between the CPU times of HGA and RSP should be noticed. HGA needs only to generate a small number of feasible chromosomes initially and during the crossover phase; chromosomes are not evaluated unless resource feasible. However, RSP has a hard time because it attempts to find a large number of resource-feasible chromosomes. Consequently, the CPU time of RSP is almost double the CPU time of HGA. In the first and second sets of problems, PGA's computation time requirements are considerably lower than HGA's. However, results are not improved significantly by letting PGA run for additional generations because PGA's convergence is much slower than HGA's, as it also includes semiactive schedules in the population.

V. CONCLUSION

We describe an HGA that exploits the advantages of both genetic search and scheduling expertise. The problem considered here is one of the most difficult scheduling problems found in the literature and subsumes a wide range of scheduling problems. We find that HGA, which includes various recently researched aspects of GA's, such as adaptive crossover and mutation probabilities, outperforms the previous attempts at solving this problem. The essential feature of HGA is the integration of problem-specific knowledge within the GA's search strategy via an indirect encoding that maps onto the heuristic space, and a solution generator that thoroughly exploits the latter encoding. The heuristic space and the solution generator incorporate a knowledge base related to the scheduling problem under consideration. Thus, HGA makes use of expert knowledge gained at a certain expense. The natural outcome of this approach is HGA's superiority in performance, as compared to a PGA, which has a general encoding applicable to any similar scheduling problem. The tradeoff between acquiring expert knowledge to achieve good solutions and applying a simple standard GA resulting in inferior solutions is a decision issue whose outcome depends on the specific circumstances creating the scheduling needs. In case the gain in achieving good results is worth the extra effort, the approach described here and similar work in the literature might lead to the concept of expert GA's (EGA's) that might be utilized for estimating future behavior of systems which are characterized by intangible features as well as tangible ones.

APPENDIX
PRIORITY RULES

1) MINSK:

$$P_j = \mathrm{lft}_j - \mathrm{est}_j - d_{jm}$$

where lft_j and est_j are the LFT and early start time (EST), respectively, of the candidate activity j; d_{jm} is the duration of activity j in its selected mode m, and P_j is the priority value of activity j. The aim of this rule is to expedite critical activities (with least total slack time), whose delay causes a prolongation of the project.

Authorized licensed use limited to: UNIVERSIDAD CARLOS III MADRID. Downloaded on June 29,2025 at 01:18:44 UTC from IEEE Xplore. Restrictions apply.

2) Minimum LFT:

$$P_j = \mathrm{lft}_j.$$

The objective of LFT is to schedule activities whose LFT's are small to decrease the risk of extending the makespan.

3) SPT:

$$P_j = d_{jm}.$$

The aim of SPT is to expedite shortest duration activities.

4) Random (RAND):

$$P_j = \text{RANDOM(ELIGSET)}.$$

where RANDOM selects an activity from among the set of candidate activities, ELIGSET.

5) Weighted resource utilization and precedence (WRUP) [40]:

$$P_j = w_p * npj + (1 - w_p) * \sum_h r_{jmh}/R_h$$

where

- w_p precedence weight $(0 \leq w_p \leq 1)$ (selected randomly);
- np_j number of immediate successors of activity j, (number of immediate predecessors in backward scheduling);
- r_{jmh} resource requirement of renewable resource type h required by activity j in mode m;
- R_h resource limit for renewable resource type h.

WRUP expedites bottleneck activities (in the project network) with many successors while attempting to pull activities with heavy resource requirements to the early phases of the project so that possible resource bottlenecks are eliminated later on when the project is running late.

6) Minimum Late Start Time (LST):

$$P_j = \mathrm{lft}_j - d_{jm}.$$

LST aims to schedule activities whose LST's are early so as to eliminate possible project delay due to postponed scheduling of critical activities.

7) Minimum EST:

$$P_j = \mathrm{est}_j.$$

EST expedites activities with the smallest EST's.

8) Minimum EFT:

$$P_j = \mathrm{est}_j + d_{jm}.$$

9) MTS:

$$P_j = \mathrm{allsons}_j$$

where $\mathrm{allsons}_j$ is the number of all successors leading from activity j. In the backward scheduling iteration, $\mathrm{allsons}_j$ becomes the number of all predecessors of activity j. The aim of MTS is to identify the activities that have the largest number of activities on the path from activity j to the ending activity of the project.

REFERENCES

[1] A. Alvarez-Valdes and J. M. Tamarit, "Heuristic algorithms for resource constrained project scheduling: A review and an empirical analysis," in *Advances in Project Scheduling*, R. Slowinski and J. Weglarz, Eds. Amsterdam, The Netherlands: Elsevier, 1989.

[2] J. C. Bean, "Genetics and random keys for sequencing and optimization," *ORSA J. Comput.*, vol. 6, pp. 154–160, 1994.

[3] J. E. Biegel and J. J. Davern, "Genetic algorithms and job-shop scheduling," *Comput. Ind. Eng.*, vol. 19, pp. 81–91, 1990.

[4] J. Blazewicz, J. K. Lenstra, and A. H. G. R. Kan, "Scheduling subject to resource constraints: Classification and complexity," *Discrete Appl. Math.*, vol. 5, pp. 11–24, 1983.

[5] F. F. Boctor, "Heuristics for scheduling projects with resource restrictions and several resource-duration modes," *Int. J. Prod. Res.*, vol. 31, pp. 2547–2558, 1994.

[6] ——, "A new and efficient heuristic for scheduling projects with resource restrictions and multiple execution modes," Working Paper, Univ. Laval, P.Q., Canada, 1994.

[7] ——, "An adaptation of the simulated annealing algorithm for solving resource constrained project scheduling problems," Working Paper, Univ. Laval, P.Q., Canada, 1994.

[8] G. A. Cleveland and S. F. Smith, "Using genetic algorithms to schedule flow shop releases," in *Proc. 3rd Int. Conf. Genetic Algorithms*. San Mateo, CA: Morgan Kaufmann, 1989.

[9] L. Davis, "Job-shop scheduling with genetic algorithms," in *Proc. Int. Conf. Genetic Algorithms Their Applicat.*, J. Grefenstette, Ed. Hillsdale, NJ: Lawrence Erlbaum, 1985.

[10] K. A. De Jong, "Adaptive system design: A genetic approach," *IEEE Trans. Syst., Man, Cybern.*, vol. SMC-10, pp. 566–574, 1980.

[11] U. Dorndorf and E. Pesch, "Evolution based learning in a job shop scheduling environment," *Comput. Oper. Res.*, vol. 22, pp. 25–40, 1995.

[12] A. Drexl and J. Grünewald, "Nonpreemptive multi-mode resource constrained project scheduling," *IIE Trans.*, vol. 25, pp. 74–81, 1993.

[13] E. Falkenauer and S. Bouffoix, "A genetic algorithm for job-shop," in *Proc. 1991 IEEE Int. Conf. Robot. Auto.*, Sacramento, CA, 1991.

[14] B. R. Fox and M. B. McMahon, "Genetic operators for sequencing problems," Planning Scheduling Group, McDonnell Douglas Space Syst., Houston, TX, 1990.

[15] D. Goldberg, *Genetic Algorithms in Search, Optimization and Machine Learning*. Reading, MA: Addison-Wesley, 1989.

[16] J. Grefenstette, "Optimization of control parameters for genetic algorithms," *IEEE Trans. Syst., Man Cybern.*, vol. SMC-16, pp. 122–128, 1986.

[17] J. W. Herrmann and C. Y. Lee, "Solving a class scheduling problem with a genetic algorithm," Dept. Ind. Syst. Eng., Univ. Florida, Working Paper, Gainesville, 1993.

[18] J. W. Herrmann, C. Y. Lee, and J. Hinchmann, "Global job-shop scheduling with a genetic algorithm," Dept. Ind. Syst. Eng., Univ. Florida, Gainesville, 1994.

[19] J. H. Holland, *Adaptation in Natural and Artificial Systems*. Ann Arbor, MI: Univ. Michigan Press, 1975.

[20] C. W. Holsapple, V. S. Jacob, P. Ramakrishnan, and J. S. Zaveri, "A genetics based hybrid scheduler for generating static schedules in flexible manufacturing context," *IEEE Trans. Syst., Man, Cybern.*, vol. 23, pp. 953–972, 1993.

[21] R. Kolisch and A. Drexl, "Local search for nonpreemptive multi-mode resource-constrained project scheduling," Christian-Albrechts-Universitat zu Kiel, Kiel, Germany, Working Paper, 1994.

[22] R. Kolisch, A. Sprecher, and A. Drexl, "Characterization and generation of a general class of resource constrained project scheduling problems," *Manage. Sci.*, vol. 41, pp. 1693–1703, 1995.

[23] D. C. Mattfeld, *Evolutionary Search and the Job-Sjop: Investigation on Genetic Algorithms for Production Scheduling*. Berlin, Germany: Physica-Verlag, 1996.

[24] G. MacMahon and D. Hadinoto, "Combining branch and bound with a genetic algorithm for job-shop scheduling problems," in *Proc. EURO Working Group on Project Management and Scheduling PMS'96*, Poznan.

[25] Z. Michalewicz, *Genetic Algorithms {+} Data Structures = Evolution Programs*. Berlin, Germany: Springer-Verlag, 1994.

[26] R. Nakano and T. Yamada, "Conventional genetic algorithms for job shop problems," in *Proc. of the Fourth International Conference on Genetic Algorithms and Their Applications*. San Diego, CA: Morgan Kaufmann, 1991.

[27] L. Özdamar and G. Ulusoy, "A local constraint based analysis approach to project scheduling under general resource constraints," *Eur. J. Oper. Res.*, vol. 79, pp. 287–298, 1994.

Authorized licensed use limited to: UNIVERSIDAD CARLOS III MADRID. Downloaded on June 29,2025 at 01:18:44 UTC from IEEE Xplore. Restrictions apply.

[28] _____, "An iterative local constraint based analysis for solving the resource constrained project scheduling problem," *J. Oper. Manage.*, vol. 14, pp. 193–208, 1996.
[29] _____, "A note on an iterative forward/backward scheduling technique with reference to a procedure by Li and Willis," *Eur. J. Oper. Res.*, vol. 89, pp. 400–407, 1996.
[30] J. H. Patterson, R. Slowinski, F. B. Talbot, and J. Weglarz, "An algorithm for a general class of precedence and resource constrained scheduling problems," in *Advances in Project Scheduling*, R. Slowinski and J. Weglarz, Eds. Amsterdam, The Netherlands: Elsevier, 1989.
[31] M. C. Portmann and F. Ghedjati, "Methodes approachee pour le probleme d'ordonnancement avec machines non identiques en paralleles et contraintes de precedence," in *Proc. Agi'94*, Poitiers, France.
[32] M. C. Portmann, "Genetic algorithms and scheduling: A state of the art and some propositions," in *Proc. Workshop Production Planning Contr.*, Mons, 1996.
[33] J. D. Schaffer, A. Caruana, L. J. Eshelman, and R. Dias, "A study of control parameters affecting on line performance of genetic algorithms for function optimization," in *Proc. 3rd Int. Conf. Genetic Algorithms*. San Mateo, CA: Morgan Kaufmann, 1989.
[34] M. Srinivas and L. M. Patnaik, "Adaptive probabilities of crossover and mutation in genetic algorithms," *IEEE Trans. Syst., Man, Cybern.*, vol. 24, pp. 656–667, 1994.
[35] A. Sprecher, S. Hartmann, and A. Drexl, "Project scheduling with discrete time resource trade-offs," Christian-Albrechts-Universitat zu Kiel, Kiel, Germany, Working Paper, 1994.
[36] T. H. Storer, S. D. Wu, and R. Vaccari, "New search spaces for sequencing problems with application to job shop scheduling," *Manage. Sci.*, vol. 38, pp. 1495–1509, 1992.
[37] G. Syswerda, "Schedule optimization using genetic algorithms," *Handbook of Genetic Algorithms*, L. Davis, Ed. New York: Van Nostrand Reinhold, 1991.
[38] F. B. Talbot, "Resource constrained project scheduling with time resource trade-offs: The nonpreemptive case," *Manage. Sci.*, vol. 28, pp. 1197–1210, 1982.
[39] S. Uckun, S. Bagchi, K. Kawamura, and Y. Miyabe, "Managing genetic search in job-shop scheduling," *IEEE Expert Mag.*, vol. 8, pp. 15–24, 1993.
[40] G. Ulusoy and L. Özdamar, "Heuristic performance and network/resource characteristics in resource constrained project scheduling," *J. Oper. Res. Soc.*, vol. 40, pp. 1145–1152, 1989.
[41] _____, "A constraint based perspective in resource constrained project scheduling," *Int. J. Prod. Res.*, vol. 32, pp. 693–705, 1994.
[42] _____, "A heuristic scheduling algorithm for improving the duration and net present value of a project," *Int. J. Oper. Prod. Manage.*, vol. 15, pp. 89–98, 1995.
[43] D. Whitley, T. Starkweather, and D. Fuquay, "Scheduling problems and travelling salesmen: The genetic edge recombination operator," in *Proc. 3rd Int. Conf. Genetic Algorithms*. San Mateo, CA: Morgan Kaufmann, 1989.
[44] D. H. Wolpert and W. G. Macready, "No free lunch theorems for search," available in the web site http://www.santafe.edu/∼wgm/papers.html, 1996.

Linet Özdamar was born in Istanbul, Turkey, in 1959. She received the B.Sc., M.Sc., and Ph.D. degrees from Boğazici University, Istanbul, in 1982, 1988, and 1991, respectively.

She was an Associate Professor in the Industrial Engineering Department, Marmara University, Istanbul, from 1992 to 1996, where she contributed to the development of the graduate and undergraduate programs in industrial engineering and engineering management. She was then with the Systems Engineering Department, Yeditepe University, Istanbul, from 1996 to 1997. She is currently the Department Head of the Computer Engineering Department in the recently founded Istanbul Kültür University. Her research interests include project management and scheduling, hierarchical production planning approaches, integration of medium and short range planning in production planning, and the use of soft computing techniques in global optimization. She has published relevant articles in international journals, such as the *European Journal of Operations Research*, *IIE Transactions*, *Computers and Operations Research*, *International Journal of Production Research*, and others.

Authorized licensed use limited to: UNIVERSIDAD CARLOS III MADRID. Downloaded on June 29,2025 at 01:18:44 UTC from IEEE Xplore. Restrictions apply.